

# Cloud computing lab

Muhammad Shoaib

**OPEN ENDED LAB**

HAJRA SARWAR

2023-BSE(5A)-022

# TERRAFORM + ANSIBLE ROLES: NGINX FRONTEND WITH 3 BACKEND HTTPD SERVERS (HA + AUTO-CONFIG)

## 1. Project Introduction

In this lab project, I designed and implemented a small multi-tier web architecture on AWS using Terraform for infrastructure provisioning and Ansible roles for automated server configuration.

The goal of this project was to **separate infrastructure and configuration concerns** in a production-like manner. Terraform was used to create all AWS resources such as VPC, subnet, security groups, and EC2 instances, while Ansible was used to configure software on those instances.

The system consists of:

- One **frontend server** running **Nginx**
- Three **backend servers** running **Apache HTTPD**

The frontend server works as a **reverse proxy and load balancer**, forwarding requests to backend servers using a **2-active + 1-backup high-availability strategy**.

All configuration is **fully automated**, so a single Terraform apply command provisions and configures everything without manual intervention.

---

## 2. Project Objectives

The main objectives of this project were:

- To design AWS infrastructure using Terraform in a clean and modular way
  - To configure frontend and backend servers using **Ansible roles**
  - To implement **Nginx load balancing with failover**
  - To integrate Terraform and Ansible so that **no manual Ansible command is required**
  - To maintain proper project structure and documentation suitable for real deployments
- 

## 3. Architecture Overview

The architecture follows a **frontend-backend model**:

- All EC2 instances are created inside **one VPC and one public subnet**
- Each instance has a public IP for testing
- SSH access is restricted to **my own public IP**
- HTTP access is allowed for testing purposes

### Load Balancer Behavior

- Two backend servers handle traffic normally (round-robin)
  - One backend server acts as **backup**
  - When the primary backends are stopped, traffic is automatically routed to the backup server
- 

## 4. Environment Setup

I performed this project using **GitHub Codespace** as my working environment.

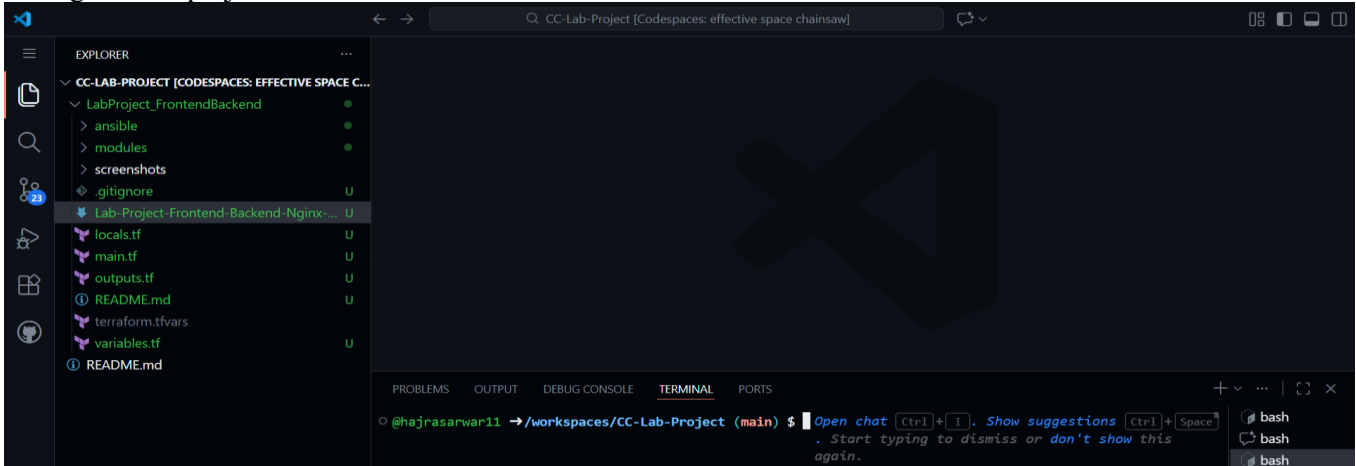
The Codespace provided a Linux environment where I installed and used:

- Terraform
- AWS CLI
- Ansible
- Python

```
Terraform: v1.14.3
AWS CLI: 1.44.24
Ansible: 2.16.3
Python: 3.12.1
```

```
✅ All tools are installed and verified!
```

I also created a GitHub repository with the required naming convention and maintained clean version control throughout the project.



## 5. Terraform Configuration – Variables and Locals

I started by defining all reusable values in **Terraform variables**. This allowed me to avoid hard-coding values and made the configuration flexible.

I defined variables for:

- VPC CIDR block
- Subnet CIDR block
- Availability zone
- Environment prefix
- Instance type
- SSH key paths



To improve security, I used **Terraform locals** to dynamically detect my public IP address and restrict SSH access only to that IP.

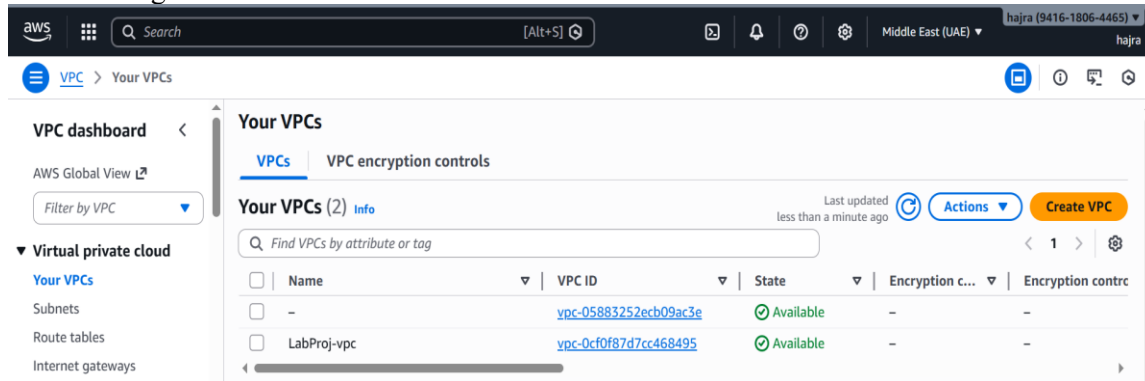


## 6. Terraform Networking Setup

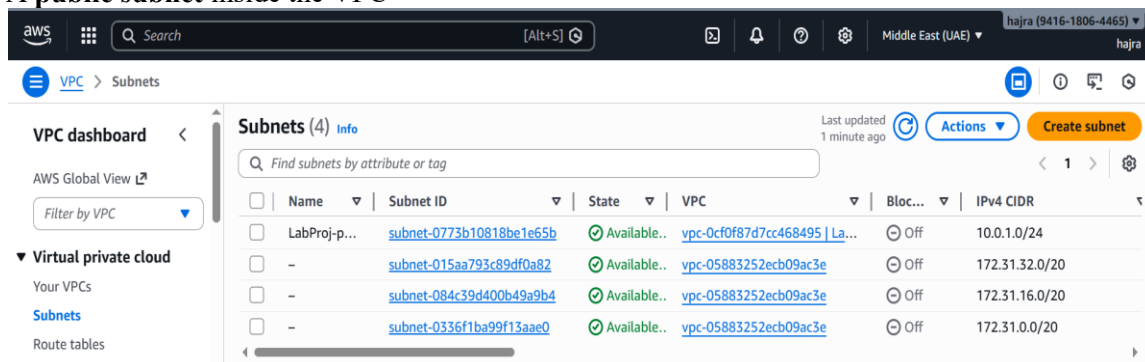
Next, I created the AWS networking components.

I provisioned:

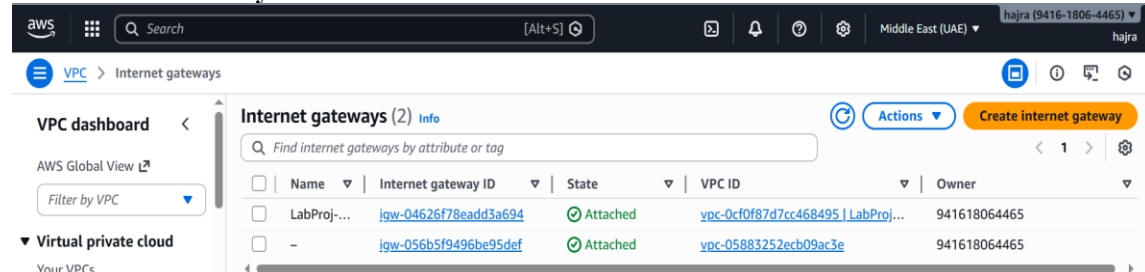
- A VPC using the CIDR block defined in variables



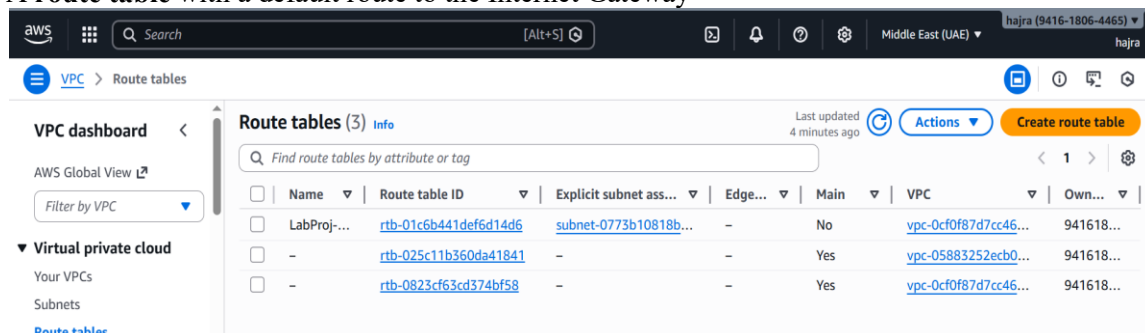
- A public subnet inside the VPC



- An Internet Gateway attached to the VPC



- A route table with a default route to the Internet Gateway



This setup ensured that all EC2 instances could communicate with the internet.

## 7. Security Group Configuration

I configured a security group with carefully scoped rules:

- SSH (port 22) allowed **only from my public IP**
- HTTP (port 80) allowed from **anywhere**

The screenshot displays the AWS Management Console interface. The top navigation bar includes the AWS logo, a search bar, and the user's account information (hajra, 9416-1806-4465). The left sidebar shows the navigation menu with categories like Capacity reservations, Images, Elastic Block Store, and Network & Security. The main content area is titled 'Security Groups (3)' and shows a list of three security groups. Below this, the 'Details' tab for the 'LabProj-sg' security group is expanded, showing its configuration and inbound rules.

**Security Groups (3)**

| Name       | Security group ID    | Security group name | VPC ID                | Description                |
|------------|----------------------|---------------------|-----------------------|----------------------------|
| -          | sg-0567ae2c936d1bb1c | default             | vpc-05883252ecb09ac3e | default VPC security group |
| LabProj-sg | sg-0f2c6db89e599ee73 | LabProj-sg          | vpc-0cf0f87d7cc468495 | Allow SSH from my IP an... |
| -          | sg-0b419a983e158f952 | default             | vpc-0cf0f87d7cc468495 | default VPC security group |

**Details for LabProj-sg**

Security group name: LabProj-sg  
 Security group ID: sg-0f2c6db89e599ee73  
 Description: Allow SSH from my IP and H TTP from anywhere  
 VPC ID: vpc-0cf0f87d7cc468495  
 Owner: 941618064465  
 Inbound rules count: 2 Permission entries  
 Outbound rules count: 1 Permission entry

**Inbound rules (2)**

| Name | Security group rule ID | IP version | Type | Protocol |
|------|------------------------|------------|------|----------|
| -    | sgr-0074dcaabc436e8b4  | IPv4       | HTTP | TCP      |
| -    | sgr-09c0555819199c465  | IPv4       | SSH  | TCP      |

This ensured secure administrative access while allowing public web access for testing.

## 8. Terraform EC2 Provisioning

After networking, I provisioned the EC2 instances.

### Frontend Instance

- One EC2 instance
- Tagged clearly as the frontend server
- Intended to run Nginx via Ansible

### Backend Instances

- Three EC2 instances created using a count-based approach
- Each backend instance received a unique tag
- These instances were later configured using the backend Ansible role

The screenshot displays the AWS Management Console interface for EC2 instances. The top navigation bar includes the AWS logo, a search bar, and the user's account information (hajra, 9416-1806-4465). The left sidebar shows the navigation menu with categories like Dashboard, AWS Global View, Events, and Instances. The main content area is titled 'Instances (4)' and shows a list of four EC2 instances. Below this, the 'Select an instance' dropdown is visible.

**Instances (4)**

| Name             | Instance ID         | Instance state | Instance type | Status check      | Alarm status  |
|------------------|---------------------|----------------|---------------|-------------------|---------------|
| LabProj-frontend | i-0cb49d3d9797eff94 | Running        | t3.micro      | 3/3 checks passed | View alarms + |
| LabProj-backend  | i-084b29708ae3dea8c | Running        | t3.micro      | 3/3 checks passed | View alarms + |
| LabProj-backend  | i-05e4f5bb213d108fd | Running        | t3.micro      | 3/3 checks passed | View alarms + |
| LabProj-backend  | i-02072643fa3f9647f | Running        | t3.micro      | 3/3 checks passed | View alarms + |

Select an instance

## 9. Terraform Outputs

To make verification easier, I defined Terraform outputs for:

- Frontend public IP
- Backend public IPs
- Backend private Ips

```
@hajrasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ terraform apply
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"
```

These outputs were later used for inventory configuration and verification.

## 10. Ansible Global Configuration

Before using roles, I configured Ansible globally.

I updated:

- `ansible.cfg` to disable host key checking and set Python interpreter

```
LabProject_FrontendBackend > ansible > ansible.cfg
1 [defaults]
2 host_key_checking = False
3 interpreter_python = /usr/bin/python3
```

- Inventory file to define frontend and backend groups

```
LabProject_FrontendBackend > ansible > inventory > hosts
1 [frontend]
2 3.28.45.9
3
4 [backends]
5 3.29.129.170
6 40.172.186.177
7 51.112.52.18
8
9 [all:vars]
10 ansible_user=ec2-user
11 ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

The inventory grouped:

- One frontend host
- Three backend hosts

## 11. Backend Role – Apache HTTPD Configuration

I created a **backend Ansible role** to configure all backend servers.

This role:

- Installed Apache HTTPD
- Enabled and started the HTTPD service

- Deployed a unique HTML page on each backend

```

@hajarasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend/ansible (main) $ a
ansible-playbook -i inventory/hosts site.yaml
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [40.172.186.177]

TASK [backend : Install httpd] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

TASK [backend : Enable and start httpd] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

TASK [backend : Deploy backend index page] *****
ok: [51.112.52.18]
changed: [40.172.186.177]
changed: [3.29.129.170]

RUNNING HANDLER [backend : Restart httpd] *****
changed: [3.29.129.170]
changed: [40.172.186.177]

PLAY RECAP *****
3.29.129.170      : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
40.172.186.177  : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
51.112.52.18    : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
  
```

Each backend page displayed:

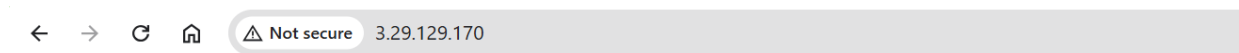
- Hostname
- Private IP address

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Backend {{ inventory_hostname }}</title>
5 </head>
6 <body>
7   <h1>Backend server: {{ inventory_hostname }}</h1>
8   <p>Private IP: {{ ansible_default_ipv4.address | default('unknown') }}</p>
9 </body>
10 </html>
  
```

This made it easy to identify which backend served a request.

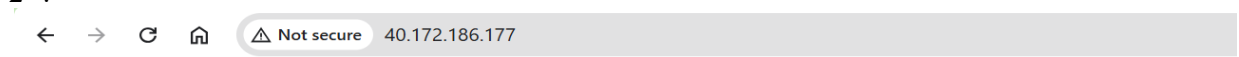
1<sup>st</sup>:



## Backend server: 3.29.129.170

Private IP: 10.0.1.155

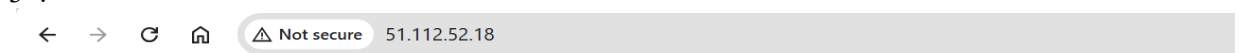
2<sup>nd</sup>:



## Backend server: 40.172.186.177

Private IP: 10.0.1.67

3<sup>rd</sup>:



## Backend server: 51.112.52.18

Private IP: 10.0.1.253

## 12. Frontend Role – Nginx Load Balancer Configuration

I then created a **frontend Ansible** role.

This role:

- Installed Nginx
- Enabled and started the Nginx service
- Deployed an Nginx configuration using a template

The screenshot displays the VS Code interface with the following components:

- Explorer Panel:** Shows the file structure of the 'CC-LAB-PROJECT [CODESPACES:...]'. The 'site.yml' file is selected.
- Problems Panel:** Empty.
- Output Panel:** Empty.
- Debug Console Panel:** Empty.
- Terminal Panel:** Shows the output of an Ansible playbook run. The command executed is `ansible-playbook -i inventory/hosts site.yml`. The output includes:
  - PLAY [Configure frontend Nginx load balancer]
  - TASK [Gathering Facts]
  - WARNING: Platform linux on host 3.28.45.9 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See [https://docs.ansible.com/ansible-core/2.16/reference\\_appendices/interpreter\\_discovery.html](https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html) for more information.
  - TASK [frontend : Install nginx]
  - TASK [frontend : Enable and start nginx]
  - TASK [frontend : Deploy nginx frontend config]
  - PLAY RECAP

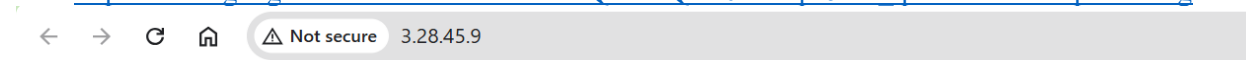
## The Nginx configuration:

- Defined an upstream with **two primary backends**
- Defined one backend as **backup**
- Used backend **private IPs**, not public Ips

```
! main.yml ../handlers U      nginx_frontend.conf.j2 U X      ! site.yml U      outputs.tf U      ! main.yml ../tasks U      LabProject_FrontendBackend > ansible > roles > frontend > templates > nginx_frontend.conf.j2
1  # Nginx frontend config template
2  user nginx;
3  worker_processes auto;
4  error_log /var/log/nginx/error.log notice;
5  pid /run/nginx.pid;
6
7  events {
8      worker_connections 1024;
9  }
10
11  http {
12      log_format main '$remote_addr - $remote_user [$time_local] "$request"
13                      '$status $body_bytes_sent "$http_referer"
14                      '"$http_user_agent" "$http_x_forwarded_for"';
15
16      access_log /var/log/nginx/access.log main;
17
18      sendfile on;
19      tcp_nopush on;
```

Frontend server:

[https://drive.google.com/file/d/1EBiKTVlAQ-zGvQaX0iELlqT6RO\\_qbElG/view?usp=sharing](https://drive.google.com/file/d/1EBiKTVlAQ-zGvQaX0iELlqT6RO_qbElG/view?usp=sharing)



**Backend server: 40.172.186.177**

Private IP: 10.0.1.67



## Backend server: 3.29.129.170

Private IP: 10.0.1.155



## 13. Ansible Main Playbook

I created a main Ansible playbook that:

- First configured all backend servers using the backend role
- Then configured the frontend server using the frontend role

```
LabProject_FrontendBackend > ansible > ! site.yaml
2 # Main Ansible playbook
3 - name: Configure backend HTTPD servers
4   hosts: backends
5   become: true
6   roles:
7     - backend
8
9 - name: Configure frontend Nginx load balancer
10  hosts: frontend
11  become: true
12  vars:
13    backend1_private_ip: "{{ hostvars[groups['backends'][0]].ansible_default_ipv4.address }}"
14    backend2_private_ip: "{{ hostvars[groups['backends'][1]].ansible_default_ipv4.address }}"
15    backup_backend_private_ip: "{{ hostvars[groups['backends'][2]].ansible_default_ipv4.address }}"
16  roles:
17    - frontend
```

Backend private IPs were dynamically retrieved using inventory data, ensuring flexibility and correctness.

## 14. Terraform and Ansible Integration

To fully automate the setup, I integrated Ansible with Terraform.

I used a Terraform **null resource** that:

- Waited for EC2 instances to be ready
- Triggered Ansible automatically
- Ensured no manual Ansible command was required

```
LabProject_FrontendBackend > main.tf
43 resource "null_resource" "ansible_config" {
44   triggers = {
45     frontend_ip = module.frontend.public_ips[0]
46     backend_ips = join(",", module.backend.public_ips)
47   }
48
49   depends_on = [
50     module.frontend,
51     module.backend
52   ]
53
54   provisioner "local-exec" {
55     command = <<-EOT
56     cd ansible
57     ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i inventory/hosts site.yaml
58     EOT
59   }
60 }
```

```
@hajarasawar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ terraform apply -auto-approve

null_resource.ansible_config (local-exec): TASK [frontend : Deploy nginx frontend config] *****
null_resource.ansible_config (local-exec): ok: [3.28.45.9]

null_resource.ansible_config (local-exec): PLAY RECAP *****
null_resource.ansible_config (local-exec): 3.28.45.9             : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
null_resource.ansible_config (local-exec): 3.29.129.170         : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
null_resource.ansible_config (local-exec): 40.172.186.177       : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
null_resource.ansible_config (local-exec): 51.112.52.18         : ok=4  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

null_resource.ansible_config: Creation complete after 18s [id=5139501220869781125]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"
```

After this integration, running Terraform apply handled **both infrastructure creation and configuration**.

## 15. System Verification

I verified the system behavior in multiple ways.

### Backend Verification

- Each backend public IP showed a unique page

1<sup>st</sup>:



**Backend server: 3.29.129.170**

Private IP: 10.0.1.155

2<sup>nd</sup>:



**Backend server: 40.172.186.177**

Private IP: 10.0.1.67

3<sup>rd</sup>:



**Backend server: 51.112.52.18**

Private IP: 10.0.1.253

### Load Balancing Verification

- Refreshing the frontend IP alternated between two backends  
[https://drive.google.com/file/d/1EBiKTVIAQ-zGvQaX0iELlqT6RO\\_qbElG/view?usp=sharing](https://drive.google.com/file/d/1EBiKTVIAQ-zGvQaX0iELlqT6RO_qbElG/view?usp=sharing)



**Backend server: 40.172.186.177**

Private IP: 10.0.1.67



**Backend server: 3.29.129.170**

Private IP: 10.0.1.155

### Failover Verification

- I stopped HTTPD on both primary backends

```
@hajrasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@3.29.129.170
Last login: Sat Jan 24 22:03:01 2026 from 20.192.21.54
[ec2-user@ip-10-0-1-155 ~]$ sudo service httpd stop
Redirecting to /bin/systemctl stop httpd.service
[ec2-user@ip-10-0-1-155 ~]$ exit
logout
Connection to 3.29.129.170 closed.
@hajrasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ ssh -i ~/.ssh/id_ed25519 ec2-user@40.172.186.177
#####
#_#
#_# Amazon Linux 2023
#_#
#_#
#_# https://aws.amazon.com/linux/amazon-linux-2023
#_#
#_#
#_#
Last login: Sat Jan 24 22:03:01 2026 from 20.192.21.54
[ec2-user@ip-10-0-1-67 ~]$ sudo service httpd stop
Redirecting to /bin/systemctl stop httpd.service
[ec2-user@ip-10-0-1-67 ~]$ exit
logout
Connection to 40.172.186.177 closed.
```

- Traffic was automatically served by the backup backend



**Backend server: 51.112.52.18**

Private IP: 10.0.1.253

## 16. Idempotence Test

I re-ran Terraform apply to verify idempotence.  
The command completed successfully:

- No errors
- No unnecessary changes
- Configuration remained consistent

```
@hajrasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ terraform apply -auto-approve
module.network.aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-0aaff9a5a405e3ecc]
module.frontend.aws_instance.this[0]: Refreshing state... [id=i-0cb49d3d9797eff94]
module.backend.aws_instance.this[1]: Refreshing state... [id=i-05e4f5bb213d108fd]
module.backend.aws_instance.this[0]: Refreshing state... [id=i-084b29708ae3dea8c]
module.backend.aws_instance.this[2]: Refreshing state... [id=i-02072643fa3f9647f]
null_resource.ansible_config: Refreshing state... [id=5139501220869781125]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_private_ips = [
  "10.0.1.155",
  "10.0.1.67",
  "10.0.1.253",
]
backend_public_ips = [
  "3.29.129.170",
  "40.172.186.177",
  "51.112.52.18",
]
frontend_public_ip = "3.28.45.9"
```

## 17. Cleanup

Finally, I destroyed all resources using Terraform to avoid unnecessary AWS charges.

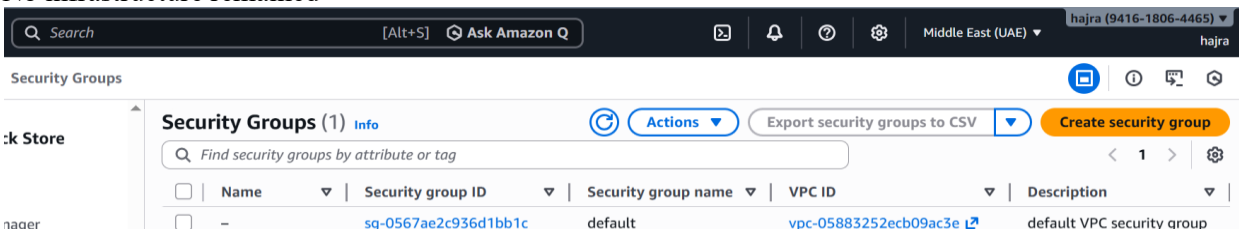
I verified that:

All EC2 instances were removed:

```
@hajrasarwar11 → /workspaces/CC-Lab-Project/LabProject_FrontendBackend (main) $ terraform destroy -auto-approve
module.backend.aws_instance.this[2]: Still destroying... [id=i-02072643fa3f9647f, 01m00s elapsed]
module.backend.aws_instance.this[0]: Destruction complete after 1m0s
module.backend.aws_instance.this[2]: Destruction complete after 1m0s
module.network.aws_subnet.public: Destroying... [id=subnet-0773b10818be1e65b]
module.network.aws_security_group.main: Destroying... [id=sg-0f2c6db89e599ee73]
module.network.aws_subnet.public: Destruction complete after 1s
module.network.aws_security_group.main: Destruction complete after 1s
module.network.aws_vpc.main: Destroying... [id=vpc-0cf0f87d7cc468495]
module.network.aws_vpc.main: Destruction complete after 0s

Destroy complete! Resources: 12 destroyed.
```

No infrastructure remained



## 18. Conclusion

In this project, I successfully implemented a **fully automated, high-availability frontend-backend system** using Terraform and Ansible roles.

This project demonstrates:

- Clean infrastructure design
- Proper role-based configuration management
- High availability and failover behavior
- Real-world automation workflow